

COMPLETE REFERENCE

NumPy Cheat Sheet

پردازش تصویر و بهینه‌سازی — NumPy کامل‌ترین مرجع سریع

```
import numpy as np
```

```
numpy ≥ 1.24
```

```
Python 3
```

```
Medical Imaging Ready
```



DATA TYPES — DTYPES

NUMPY DTYPES

```
np.int8/16/32/64
```

عدد صحیح علامت‌دار

```
np.uint8/16/32/64
```

عدد صحیح بدون علامت (تصویر!)

```
np.float16/32/64
```

اعداد اعشاری

```
np.complex64/128
```

اعداد مختلط

```
np.bool_
```

True/False منطقی

```
np.str_
```

Unicode رشته

استفاده می‌شود `float32` یا `uint16` معمولاً، CT، پردازش تصویر



ARRAY CREATION — ساخت آرایه

از داده مستقیم

<code>np.array([1,2,3])</code>	لیست از Python
<code>np.array([[1,2],[3,4]])</code>	ماتریس ۲×۲
<code>np.array([1,2], dtype=np.float32)</code>	مشخص dtype با
<code>np.asarray(data)</code>	بدون کپی در صورت امکان
<code>np.frombuffer(buf, dtype)</code>	بایتری buffer از
<code>np.fromfile('f.bin', dtype)</code>	از فایل بایتری

آرایه‌های خاص

<code>np.zeros((3,4))</code>	همه صفر
<code>np.ones((3,4))</code>	همه یک
<code>np.full((3,4), 7)</code>	پر با مقدار دلخواه
<code>np.eye(n)</code>	n×n ماتریس واحد
<code>np.identity(n)</code>	eye مشابه
<code>np.empty((3,4))</code>	بدون مقداردهی اولیه (سریع)
<code>np.zeros_like(a)</code>	a صفر با شکل
<code>np.ones_like(a)</code>	a یک با شکل
<code>np.full_like(a, v)</code>	a با شکل v مقدار

دنباله‌ها و بازدها

<code>np.arange(10)</code>	0 تا 9
<code>np.arange(2, 20, 3)</code>	start, stop, step

<code>np.linspace(0,1,100)</code>	نقطه یکنواخت [0,1] ۱۰۰
<code>np.logspace(0,3,4)</code>	لگاریتمی: 1,10,100,1000
<code>np.geomspace(1,1000,4)</code>	تصادد هندسی

NP.RANDOM — تصادفی	
<code>np.random.seed(42)</code>	seed تنظیم
<code>rng = np.random.default_rng(42)</code>	جدید (توصیه شده) Generator
<code>rng.random((3,4))</code>	uniform [0,1)
<code>rng.integers(0,10,(3,4))</code>	صحیح تصادفی
<code>rng.normal(0,1,(3,4))</code>	نرمال (μ, σ)
<code>rng.choice(arr, n)</code>	انتخاب تصادفی
<code>rng.shuffle(arr)</code>	inplace درهم آمیختن
<code>rng.permutation(arr)</code>	درهم آمیختن کپی

ARRAY ATTRIBUTES — ویژگی‌های آرایه

خواص اصلی	
<code>a.shape</code>	(3, 4, 5)
<code>a.ndim</code>	3
<code>a.size</code>	(کل عناصر) 60
<code>a.dtype</code>	float64
<code>a.itemsize</code>	(بایت/عنصر) 8

<code>a.nbytes</code>	480 (کل بایت)
<code>a.T</code>	ترانپوز
<code>a.flat</code>	خطی <code>iterator</code>
<code>a.strides</code>	گام‌های حافظه

نمایش محورها (AXIS)	
1D — axis=0 <code>[1, 2, 3, 4]</code> → ↓	2D — axis=0 (سطر) <code>[[1,2],</code> <code>[3,4]]</code> ↓
2D — axis=1 (ستون) <code>[[1,2],</code> <code>[3,4]]</code> →	3D (C,H,W) — CT <code>axis0=Channel</code> <code>axis1=Height</code> <code>axis2=Width</code>

 INDEXING & SLICING — ایندکس‌گذاری

BASIC INDEXING	
<code>a[0]</code>	اولین عنصر
<code>a[-1]</code>	آخرین عنصر
<code>a[2,3]</code>	سطر ۲، ستون ۳
<code>a[1:4]</code>	عناصر ۱ تا ۳
<code>a[:,2]</code>	یک‌درمیان
<code>a[:, -1]</code>	معکوس
<code>a[1:3, 2:5]</code>	زیرماتریس
<code>a[:, 0]</code>	ستون اول

<code>a[0, :]</code>	سطر اول
----------------------	---------

ADVANCED INDEXING

<code>a[[0,2,4]]</code>	fancy indexing (ایکپی)
<code>a[a > 5]</code>	boolean masking
<code>a[np.where(a>5)]</code>	به عنوان ایندکس where
<code>np.where(a>5, a, 0)</code>	جایگزینی شرطی
<code>np.take(a, [0,2], axis=1)</code>	انتخاب ستون
<code>np.ix_([0,1],[2,3])</code>	شبکه ایندکس
<code>np.nonzero(a)</code>	ایندکس عناصر غیر صفر
<code>np.argwhere(a > 0)</code>	True مختصات عناصر



RESHAPING & MANIPULATION — تغییر شکل

تغییر ابعاد

<code>a.reshape(2,6)</code>	بدون کپی در صورت امکان
<code>a.reshape(-1, 3)</code>	محاسبه خودکار = -1
<code>a.ravel()</code>	تسطیح (اگر ممکن view)
<code>a.flatten()</code>	تسطیح (همیشه کپی)
<code>a.T</code>	ترانپوز
<code>np.transpose(a, (2,0,1))</code>	جابجایی محورها دلخواه
	افزودن بُعد جدید

<code>a[np.newaxis, :]</code>	
<code>np.expand_dims(a, axis=0)</code>	newaxis مشابه
<code>np.squeeze(a)</code>	حذف ابعاد یکتایی

ترکیب و تقسیم	
<code>np.concatenate([a,b], axis=0)</code>	الحاق در محور دلخواه
<code>np.vstack([a,b])</code>	(axis=0) عمودی
<code>np.hstack([a,b])</code>	(axis=1) افقی
<code>np.stack([a,b], axis=0)</code>	محور جدید
<code>np.dstack([a,b])</code>	(axis=2) عمقی
<code>np.split(a, 3)</code>	تقسیم مساوی
<code>np.split(a, [2,5])</code>	در نقاط مشخص
<code>np.vsplit(a, 2)</code>	تقسیم عمودی
<code>np.hsplit(a, 2)</code>	تقسیم افقی

+ MATH OPERATIONS — عملیات ریاضی

(ELEMENTWISE) عملیات عنصری	
<code>a + b / np.add(a,b)</code>	جمع
<code>a - b / np.subtract</code>	تفریق
<code>a * b / np.multiply</code>	ضرب عنصری
<code>a / b / np.divide</code>	تقسیم

<code>a // b</code>	تقسیم صحیح
<code>a % b</code>	باقیمانده
<code>a ** b / np.power</code>	توان
<code>np.sqrt(a)</code>	ریشه دوم
<code>np.abs(a)</code>	قدر مطلق
<code>np.sign(a)</code>	علامت: 1+, 0, 1-
<code>np.clip(a, 0, 1)</code>	کلمپ در [0,1]

توابع ریاضی	
<code>np.exp(a)</code>	e^x
<code>np.exp2(a)</code>	2^x
<code>np.log(a)</code>	$\ln(x)$
<code>np.log2(a)</code>	$\log_2(x)$
<code>np.log10(a)</code>	$\log_{10}(x)$
<code>np.sin/cos/tan(a)</code>	مثلثاتی (رادیان)
<code>np.arcsin/arccos/arctan</code>	معکوس مثلثاتی
<code>np.degrees(a)</code>	رادیان $\rightarrow$ درجه
<code>np.radians(a)</code>	درجه $\rightarrow$ رادیان
<code>np.ceil/floor/round(a)</code>	گرد کردن
<code>np.hypot(a,b)</code>	$\sqrt{a^2+b^2}$

توابع تجمیعی	
<code>np.sum(a, axis=0)</code>	مجموع

<code>np.prod(a)</code>	ضرب همه
<code>np.cumsum(a)</code>	مجموع تجمیعی
<code>np.cumprod(a)</code>	ضرب تجمیعی
<code>np.mean(a)</code>	میانگین
<code>np.median(a)</code>	میانه
<code>np.std(a)</code>	انحراف معیار
<code>np.var(a)</code>	واریانس
<code>np.min/max(a)</code>	کمینه / بیشینه
<code>np.argmin/argmax(a)</code>	ایندکس کمینه/بیشینه
<code>np.ptp(a)</code>	$\text{range} = \text{max} - \text{min}$
<code>np.percentile(a, 75)</code>	صدک ۷۵م
<code>np.quantile(a, 0.75)</code>	percentile مشابه

مقایسه و منطق

<code>a == b / np.equal</code>	برابری عنصری
<code>a > b / np.greater</code>	بزرگتر
<code>np.logical_and(a,b)</code>	منطقی AND
<code>np.logical_or(a,b)</code>	منطقی OR
<code>np.logical_not(a)</code>	منطقی NOT
<code>np.any(a)</code>	هست؟ True آیا هیچ
<code>np.all(a)</code>	هستند؟ True آیا همه
<code>np.isnan(a)</code>	NaN بررسی
<code>np.isinf(a)</code>	Inf بررسی
<code>np.isfinite(a)</code>	مقدار محدود؟

<code>np.allclose(a,b)</code>	تقریباً برابر؟
<code>np.array_equal(a,b)</code>	دقیقاً برابر؟

1234

LINEAR ALGEBRA — NP.LINALG

عملیات ماتریسی	
<code>a @ b</code>	ضرب ماتریسی
<code>np.matmul(a, b)</code>	@ مشابه
<code>np.dot(a, b)</code>	ضرب داخلی
<code>np.inner(a, b)</code>	ضرب داخلی برداری
<code>np.outer(a, b)</code>	ضرب خارجی
<code>np.cross(a, b)</code>	Dضرب برداری ۳
<code>np.kron(a, b)</code>	ضرب کرونکر
<code>np.tensordot(a,b,axes)</code>	ضرب تانسوری
<code>np.einsum('ij,jk→ik',a,b)</code>	Einstein summation

توابع NP.LINALG	
<code>np.linalg.det(a)</code>	دترمینان
<code>np.linalg.inv(a)</code>	معکوس ماتریس
<code>np.linalg.pinv(a)</code>	Moore-Penrose شبه‌معکوس
<code>np.linalg.eig(a)</code>	مقادیر/بردارهای ویژه
<code>np.linalg.eigh(a)</code>	ویژه برای ماتریس متقارن

<code>np.linalg.svd(a)</code>	SVD تجزیه
<code>np.linalg.norm(a)</code>	(L2: پیش‌فرض) نرم
<code>np.linalg.norm(a, ord=1)</code>	L1 نرم
<code>np.linalg.solve(A, b)</code>	$Ax=b$ حل
<code>np.linalg.lstsq(A, b)</code>	کمترین مربعات
<code>np.linalg.matrix_rank(a)</code>	رتبه ماتریس
<code>np.linalg.trace(a)</code>	اثر (مجموع قطر)
<code>np.linalg.cholesky(a)</code>	Cholesky تجزیه
<code>np.linalg.qr(a)</code>	QR تجزیه



BROADCASTING — قوانین انبساط

BROADCASTING قوانین

قانون: ابعاد از راست مقایسه می‌شوند # سازگارند اگر: برابرند یا یکی #
`a = np.ones((3, 1, 5)) # shape (3, 1, 5) b = np.ones((4, 5)) # shape (4, 5) c = a + b # shape (3, 4, 5) ✓` مثال #
 per-feature: عملی - نرمالیزاسیون `X = np.random.randn(100, 20) # 100 sample, 20 feature mu = X.mean(axis=0) # shape (20,) std = X.std(axis=0) # shape (20,) X_norm = (X - mu) / std # broadcast`
 broadcast خودکار `(100,20) →` row = `np.array([1,2,3]) # (3,) col = np.array([10,20])[:,np.newaxis] # (2,1) result = row + col # (2,3)` صحیح



SORTING & SEARCHING — مرتب‌سازی

<code>np.sort(a)</code>	کپی مرتب‌شده
<code>a.sort()</code>	مرتب‌سازی درجا
<code>np.sort(a, axis=0)</code>	مرتب‌سازی ستونی
<code>np.argsort(a)</code>	ایندکس‌های مرتب‌سازی
<code>np.argsort(a)[::-1]</code>	نزولی
<code>np.lexsort((b, a))</code>	مرتب بر چند کلید
<code>np.partition(a, k)</code>	k-th smallest (سریع)
<code>np.argpartition(a, k)</code>	partition ایندکس
<code>np.searchsorted(a, v)</code>	جای درج در آرایه مرتب

جستجو و فیلتر

<code>np.where(cond, x, y)</code>	انتخاب شرطی
<code>np.select([c1,c2],[v1,v2])</code>	چند شرط
<code>np.extract(cond, a)</code>	فیلتر با شرط
<code>np.compress(mask, a, axis)</code>	mask فشردسازی با
<code>np.unique(a)</code>	مقادیر یکتا
<code>np.unique(a, return_counts=True)</code>	با تعداد تکرار
<code>np.in1d(a, b)</code>	هست؟ b در a آیا
<code>np.intersect1d(a, b)</code>	اشتراک
<code>np.union1d(a, b)</code>	اتحاد
<code>np.setdiff1d(a, b)</code>	a - b تفاضل

NP.FFT

<code>np.fft.fft(a)</code>	D تبدیل فوریه 1
<code>np.fft.ifft(a)</code>	D معکوس فوریه 1
<code>np.fft.fft2(a)</code>	(تصویر) D فوریه 2
<code>np.fft.ifft2(a)</code>	D معکوس 2
<code>np.fft.fftn(a)</code>	بُعدی n فوریه
<code>np.fft.rfft(a)</code>	real-valued فوریه
<code>np.fft.fftshift(a)</code>	به مرکز DC جابجایی
<code>np.fft.fftfreq(n, d)</code>	فرکانس‌های FFT

CT تصویر FFT مثال

```
import numpy as np # CT اسلایس
float ct_slice = np.random.rand(512, 512).astype('float32') # D تبدیل فوریه 2
F = np.fft.fft2(ct_slice) F_shifted = np.fft.fftshift(F) # طیف مگنیتود
magnitude = np.log(1 + np.abs(F_shifted)) # فیلتر (برای نمایش log)
H, W = ct_slice.shape mask = np.zeros((H, W))
cy, cx = H//2, W//2 R = 50 Y, X = np.ogrid[:H, :W] mask[(Y-cy)**2 + (X-cx)**2 <= R**2] = 1
filtered = np.fft.ifft2(np.fft.ifftshift(F_shifted * mask)).real
```



<code>np.cov(x, y)</code>	ماتریس کوواریانس
<code>np.corrcoef(x, y)</code>	ضریب همبستگی پیرسون
<code>np.histogram(a, bins=20)</code>	هیستوگرام
<code>np.histogram2d(x, y)</code>	D هیستوگرام 2
<code>np.bincount(a)</code>	تعداد عناصر صحیح
<code>np.digitize(x, bins)</code>	بندی bin
<code>np.average(a, weights=w)</code>	میانگین وزنی
<code>np.nanmean(a)</code>	NaN میانگین بدون
<code>np.nanstd(a)</code>	std بدون NaN
<code>np.nansum(a)</code>	NaN مجموع بدون

توزیع‌های تصادفی

<code>rng.normal(μ, σ, size)</code>	توزیع نرمال
<code>rng.uniform(low, high, size)</code>	یکنواخت
<code>rng.binomial(n, p, size)</code>	دوجمله‌ای
<code>rng.poisson(λ, size)</code>	پواسون
<code>rng.exponential(β, size)</code>	نمایی
<code>rng.beta(a, b, size)</code>	بتا
<code>rng.gamma(k, θ, size)</code>	گاما
<code>rng.dirichlet(α, size)</code>	دیریکله
<code>rng.multivariate_normal(μ, Σ)</code>	نرمال چندمتغیره



ذخیره و بارگذاری

<code>np.save('f.npy', a)</code>	ذخیره یک آرایه
<code>np.load('f.npy')</code>	.npy بارگذاری
<code>np.savez('f.npz', a=a, b=b)</code>	ذخیره چند آرایه
<code>np.savez_compressed('f.npz', ...)</code>	فشرده‌شده
<code>d = np.load('f.npz')</code> <code>d['a']</code>	npz خواندن از
<code>np.savetxt('f.csv', a, delimiter=',')</code>	ذخیره CSV
<code>np.loadtxt('f.csv', delimiter=',')</code>	CSV خواندن
<code>np.genfromtxt('f.csv', delimiter=',', names=True)</code>	CSV با هدر

مدیریت حافظه

<code>np.copy(a)</code>	کپی کامل
<code>a.copy()</code>	مشابه
<code>a.view()</code>	view (بدون کپی)
<code>np.shares_memory(a, b)</code>	آیا حافظه مشترک؟
<code>a.base</code>	view اگر آرایه اصلی
<code>np.ascontiguousarray(a)</code>	C-contiguous حافظه
<code>np.asfortranarray(a)</code>	Fortran-contiguous حافظه
<code>a.flags</code>	وضعیت حافظه

⚠ Slicing یک **view** می‌سازد! تغییر در آن اصل را هم تغییر می‌دهد.



MEDICAL IMAGING — کاربردهای CT / پردازش تصویر

CT SCAN عملیات رایج در پردازش

```
import numpy as np # به HU [0,1] نرمالیزاسیون
def normalize_ct(volume, hu_min=-1000, hu_max=400): volume = np.clip(volume, hu_min, hu_max) return (volume - hu_min) / (hu_max - hu_min) # — Window/Level (Lung Window)
def apply_window(img, center=-600, width=1500): low = center - width / 2 high = center + width / 2 return np.clip(img, low, high) # — Extract 3D patches برای nodule detection
def extract_patch_3d(volume, center, size=64): z, y, x = center h = size // 2 return volume[z-h:z+h, y-h:y+h, x-h:x+h] # — Binary mask و اعمال به حجم
mask = (ct_volume > -500) & (ct_volume < -100) # ریه lung = ct_volume * mask # — آمار هر نودول
nodule_region = ct_volume[mask] stats = { 'mean': np.mean(nodule_region), 'std': np.std(nodule_region), 'p10': np.percentile(nodule_region, 10), 'p90': np.percentile(nodule_region, 90), 'volume_voxels': np.sum(mask) } # — Data augmentation با flip/rotate
aug_flip_h = np.flip(patch, axis=2) # horizontal aug_flip_v = np.flip(patch, axis=1) # vertical aug_rot90 = np.rot90(patch, k=1, axes=(1,2)) # rotate slice # — Batch برای مدل
batch = np.stack([p1, p2, p3], axis=0) # (N, D, H, W) batch = batch[:, np.newaxis, ...] # (N,1,D,H,W) for PyTorch batch_f32 = batch.astype(np.float32) / 255.0
```



ADVANCED — پیشرفته

STRUCTURED ARRAYS

```
dt = np.dtype([ ('patient_id', 'U20'), ('age', np.int16),
                ('nodule_size', np.float32), ('malignant', np.bool_) ]) data =
np.zeros(100, dtype=dt) data[0]['patient_id'] = 'P001' data[0]
['nodule_size'] = 5.2 # فیلتر موارد بدخیم mal =
data[data['malignant'] = True]
```

MASKED ARRAYS & POLYNOMIALS

```
import numpy.ma as ma # پوشاندن مقادیر نامعتبر masked =
ma.masked_where(a < 0, a) masked = ma.masked_invalid(a) # NaN & Inf
valid_mean = masked.mean() # ————— چند جمله ای ————— p
= np.poly1d([1, -3, 2]) #  $x^2-3x+2$  roots = np.roots([1,-3,2]) coeffs
= np.polyfit(x, y, deg=3) y_fit = np.polyval(coeffs, x)
```

VECTORIZED & APPLY

```
# برداری سازی تابع Python def hu_to_label(hu): if hu < -500: return
0 # هوا elif hu < 100: return 1 # بافت نرم else: return 2 # استخوان
vfunc = np.vectorize(hu_to_label) labels = vfunc(ct_volume) #
np.apply_along_axis result = np.apply_along_axis( np.mean, axis=0,
arr=data )
```

MESHGRID & COORDINATES

```
x = np.linspace(-1, 1, 100) y = np.linspace(-1, 1, 100) XX, YY =
np.meshgrid(x, y) Z = np.sqrt(XX**2 + YY**2) # بهینه تر با
ogrid/mgrid: Y, X = np.ogrid[0:512, 0:512] # دایره برای lung nodule
mask: cy, cx, r = 256, 256, 30 circle = (Y-cy)**2 + (X-cx)**2 ≤
```


ایندکس `idx = np.indices((3,4)) # grid` نمونه برداری چندبُعدی `r**2` #



PERFORMANCE TIPS — بهینه‌سازی

نکات کلیدی بهینه‌سازی

پرهیز کن `for` از حلقه

`dtype=float32` در DL (نه `float64`)

برای ضرب تانسوری `np.einsum`

`view` به جای `copy`

وقتی پر می‌کنی `np.zeros` به جای `np.empty`

`cache` سرعت → `contiguous memory`

`np.partition` به جای `sort` برای `top-k` کامل

`tile/repeat` به جای `broadcasting`

`allocation` پارامتر برای کاهش `out=`

`memmap` برای فایل‌های بزرگ (حجم CT)

```
# سریع - Python result = [x**2 + 1 for x in arr] # ✓  
vectorized result = arr**2 + 1 # ✓  
np.add(a, b, out=c) # ✓  
Memory-mapped برای CT volume =  
np.memmap('ct_scan.dat', dtype='float32', mode='r', shape=  
(512,512,300)) # ✓  
einsum - batch ضرب # a: (B,M,K) b:  
(B,K,N) → (B,M,N) result = np.einsum('bmk,bkn→bmn', a, b)
```



QUICK REFERENCE — خلاصه سریع

تبدیل نوع

<code>a.astype(np.float32)</code>	تبدیل dtype
<code>a.astype(int)</code>	به int
<code>a.tolist()</code>	به list Python
<code>a.tobytes()</code>	خام bytes به

<code>float(a)</code>	scalar به float
<code>a.item()</code>	numpy scalar → Python

ML توابع پرکاربرد در	
<code>np.dot(W, x) + b</code>	Linear لایه
<code>1 / (1 + np.exp(-x))</code>	Sigmoid
<code>np.maximum(0, x)</code>	ReLU
<code>np.tanh(x)</code>	Tanh
<code>e = np.exp(x - x.max())</code> <code>e / e.sum()</code>	Softmax (stable)
<code>-np.mean(y*np.log(p+1e-9))</code>	Cross-Entropy Loss
<code>np.mean((y - y_hat)**2)</code>	MSE Loss

ثابت‌ها	
<code>np.pi</code>	$\pi = 3.14159...$
<code>np.e</code>	$e = 2.71828...$
<code>np.inf</code>	∞
<code>np.nan</code>	Not a Number
<code>np.newaxis</code>	None (alias)
<code>np.finfo(np.float32).eps</code>	ماشین epsilon
<code>np.iinfo(np.int16).max</code>	int16 بیشترین

DEBUG & INSPECT

<code>a.shape, a.dtype</code>	بررسی سریع
<code>np.info(np.sum)</code>	مستندات تابع
<code>np.set_printoptions(precision=3)</code>	دقت نمایش
<code>np.set_printoptions(suppress=True)</code>	علمی notation بدون
<code>np.testing.assert_allclose(a,b)</code>	تست عددی
<code>%timeit np.dot(a,b)</code>	اندازه‌گیری زمان